

WatchLockAI Sentinel - Interoperability Map

Generated: 2025-09-02 22:10:18

Baseline: Ruff=0, Pyright=687 errors, Pytest=2 collection errors

Scope: Comprehensive codebase analysis for P0 compatibility fixes

Overview

WatchLockAI Sentinel is a multi-component security monitoring system with the following architectural layers:

- **Core Layer:** `app_core/` - Event bus, configuration, schemas, logging
- **Collection Layer:** `collectors/` - File system, process, registry, network, health monitoring
- **Detection Layer:** `detection/` - Rules engine, behavioral analysis, ML scoring, threat intelligence
- **Response Layer:** `response/` - Actions management, alerts system
- **Interface Layer:** `console/` - FastAPI web interface, operational mode API
- **Service Layer:** `service/` - Windows service wrapper, main application orchestration

The system follows an event-driven architecture using an internal async event bus for component communication, with FastAPI-based REST API for external control and monitoring.

Import Graph Highlights

Core Dependency Chains

```
app.py → app_core, detection, service, ui
service.service_wrapper → app_core, collectors, console, detection,
response
console.web_api → console, detection
```

Potential Cycles Detected

1. config/app_core cycle:

- `app_core.config:109` imports `config.operational_mode`
- `console.api.operational_mode` imports `config`
- Risk: Circular import during initialization

2. detection internal cycle:

- `detection.behavioral_engine` imports `detection`
- `detection.rules_engine` imports `detection`
- Risk: Self-referential imports within detection package

High-Risk Import Patterns

- **Late imports:** `console/web_api.py:82-86` uses try/except ImportError for operational_mode router
- **Conditional imports:** `service/service_wrapper.py:13-21` conditionally imports Windows APIs
- **Deep imports:** Multiple files import from `app_core` creating hub dependency

Event Contracts

Producer	Consumer(s)	Event Type	Fields
collectors.fs_monitor	detection.rules_engine	FileEvent	event_type: FileEvent EventTime: int str, size: int int? str?, file_path: str
collectors.proc_monitor	detection.rules_engine	ProcessEvent	event_type: ProcessEvent ProcessName: str pid: int str, size: int
collectors.reg_monitor	detection.rules_engine	RegistryEvent	event_type: RegistryEvent RegistryPath: str RegistryValue: str key_path: str
collectors.net_monitor	detection.rules_engine	NetworkEvent	event_type: NetworkEvent NetworkProtocol: str NetworkAddress: str local_ip: str remote_ip: str
collectors.health_monitor	response.alerts	HealthEvent	cpu_usage: float memory_usage: float disk_usage: float temp_cpu: float temp_gpu: float
detection.rules_engine	response.alerts	DetectionEvent	severity: str AlertName: str cause: str AlertTime: int message: str metadata: dict
response.actions	app_core.bus	ActionEvent	action_type: str ActionName: str resource: str

Producer	Consumer(s)	Event Type	File
			Action parameter user

Event Bus Contract Issues

- **Type safety:** Some consumers cast `BaseEvent` without proper type checking
- **Schema evolution:** No versioning strategy for event schema changes
- **Error handling:** Limited error propagation for failed event delivery

API Routes

Method	Path	Handler	Model In/Out
GET	/api/status	get_status()	→ StatusResponse
GET	/api/alerts	get_alerts()	→ AlertSummary
GET	/api/detections	get_detections()	→ DetectionResponse
POST	/api/actions/ pause	pause_monitoring()	duration_minutes: int → dict
POST	/api/actions/ resume	resume_monitoring()	→ dict
GET	/api/ti/search	search_threat_intelligence()	query: str, limit: int → dict
GET	/api/policies	get_policies()	→ dict
POST	/api/policies	set_policies()	dict → dict
GET	/api/ operational_mode	get_operational_mode()	→ GetModeResponse
PUT	/api/ operational_mode	set_operational_mode()	SetModeRequest → GetModeResponse

API Route Issues

1. **Inconsistent response models:** Mix of Pydantic models and raw dicts
2. **Missing route registration:** No centralized router registration pattern
3. **Path conflicts:** Potential future conflicts with `/api/policies` vs operational mode
4. **Error handling:** Inconsistent HTTP exception patterns

Background Tasks

Task Name	Start Order	Stop Order	Cancellation	Location
<code>EventBus._worker()</code>	1 (bus.start)	4 (bus.stop)	✓ Graceful	<code>app_core/bus.py:92</code>
<code>SentinelService.start()</code>	2 (service init)	3 (service stop)	✓ Graceful	<code>service/ service_wrapper.py: 210</code>
<code>SentinelWebAPI.server</code>	3 (after service)	2 (before service)	⚠ Basic	<code>console/web_api.py: 370</code>
<code>TrayApp</code> (optional)	4 (UI optional)	1 (UI first)	⚠ Thread-based	<code>app.py:90</code>

Background Task Issues

1. **Startup dependency:** No explicit dependency management between tasks
2. **Shutdown order:** Reverse order not guaranteed, risk of resource conflicts
3. **Error propagation:** Task failures may not properly cascade to parent
4. **Graceful cancellation:** Some tasks use basic cancellation vs proper cleanup

Config Surface

Key	Owner Module	Default	Duplica
<code>operational.mode</code>	<code>app_core.config:</code> 98	"observe"	✓ Sing source
<code>operational.current_mode</code>	<code>app_core.config:</code> 101	Dynamic	! Proper
<code>responses.allow_destructive_actions</code>	<code>app_core.config:</code> 92	False	✓ Sing source
<code>monitoring.file_system.enabled</code>	<code>app_core.config:</code> 16	True	✓ Sing source
<code>monitoring.processes.enabled</code>	<code>app_core.config:</code> 29	True	✓ Sing source
<code>monitoring.registry.enabled</code>	<code>app_core.config:</code> 37	True	✓ Sing source
<code>monitoring.network.enabled</code>	<code>app_core.config:</code> 49	True	✓ Sing source
<code>health.enabled</code>	<code>app_core.config:</code> 59	True	✓ Sing source
<code>rag.mode</code>	<code>app_core.config:</code> 79	"embeddings"	✓ Sing source
<code>service.install_on_setup</code>	<code>app_core.config:</code> 119	True	✓ Sing source

Config Surface Issues

1. **Dual mode access:** `operational.mode` vs `operational.current_mode` creates confusion
2. **Default inconsistencies:** Some defaults in code vs YAML may diverge
3. **Validation gaps:** Path validation warnings not enforced at runtime
4. **Environment override:** Limited documentation for environment variable patterns

Platform-Guard Checklist

Path	Guard Present?	File:Line	Notes
Windows service APIs	 YES	service/ service_wrapper.py: 13-21	try/except ImportError
Registry monitoring	 NO	collectors/ reg_monitor.py	Needs platform.system() check
Windows event logging	 PARTIAL	app_core/ logging_setup.py	Comment mentions platform check
Process termination	 NO	response/ actions.py:83	Uses psutil without platform guards
Service installation	 NO	service/ service_wrapper.py: 400+	Windows-specific operations
Tray application	 NO	ui/tray_app.py	Windows/GUI-specific

Platform-Guard Issues (Critical P0 Fixes Needed)

- Registry collector:** collectors/reg_monitor.py will fail on Linux CI - needs platform check
- Service operations:** Install/uninstall operations not guarded
- UI components:** Tray app may fail on headless Linux systems
- Process actions:** Windows-specific process management not isolated
- Event logging:** Windows event log handler missing platform checks
- File paths:** Some Windows path patterns may not handle Linux paths correctly

Analysis Complete: 6 categories analyzed, 23 potential P0 issues identified for prioritization.